

暗室藏书 · PROJECT ORIGIN

项目起源

从 2026 年 5 月的个人构想，到 7 月课程落地与暑假工程化
记录产品方向、实现演进与公开基线

一、项目定位

暗室藏书不是把图书表格换成另一种颜色，而是把“发现、借阅、归还、补货、审核和审计”放在同一条可追溯链路里。读者端负责进入、寻找、停留与回望；管理端负责维护、审核、统计与协作。两者共享同一套纸墨与灯火语言，但不强行使用同一种密度。

项目的边界是单机或小团队部署的增强型 Spring Boot 单体。MySQL 保存最终业务事实，Redis、RabbitMQ 和邮件是可关闭的增强能力。这样既保留真实系统需要的可靠性，也不把课程项目堆成无法解释的微服务集合。

当前交付基线：Spring Boot 3.5.16、MyBatis-Plus 3.5.17、Vue 3.5.40、Vite 8.1.5、MySQL 8。后端监听 20606，前端监听 5175。

二、项目缘起与时间线

项目想法形成于 2026 年 5 月。最初关注的不是“完成一个课程管理系统”，而是怎样让图书馆软件同时具备清楚的业务秩序、可解释的角色分工，以及区别于通用后台模板的阅读氛围。暗室、灯火、纸墨和封蜡红的视觉表达，以及读者、馆务、采购和物流之间的协作，都从这次构想逐步展开。

2026 年 7 月的大二短学期提供了落地窗口。教师发放的教学底座只有普通登录、用户和公告等早期条件，连图书增删改查都没有形成完整闭环；作者借课程时间实现 5 月构想，并在课程提交后继续补齐业务、迁移前端、升级后端、重构一致性和建立验证体系。归档底座与当前仓库对比表明，当前版本的技术栈、包名、数据库、角色模型和业务范围均已形成独立工程。

项目早期被当作一次性 demo，Git 在重构中途才加入。公开仓库不伪造更早提交；首次公开提交只表示经过审查的当前基线。公开材料只保留明确标注的本地演示账号；真实账号凭据、数据库密码、邮箱授权码和生产密钥均不得入库。

三、角色与业务闭环

角色	主要职责	边界
超级管理员	全局用户、权限、文件、采购物流和审计	保留系统最终控制权
馆务协调员	馆藏、借阅、公告、内容审核和采购需求	不能修改同级或更高权限账号
读者	检索、借还、预约、收藏和社区互动	只能操作自己的业务数据
采购员	认领采购单、推进采购、分配物流	不能管理用户和馆藏基础数据
物流员	运输、到馆、入库和库存补充	只能处理分配给自己的物流任务

核心链路从读者查询图书开始，经借阅、归还、库存恢复和预约通知形成流通闭环；低库存告警再进入采购、物流、到馆和入库，形成供应闭环。书评、回复、举报、审核和操作日志构成内容治理闭环。



图 1 读者端馆藏检索与虚构书目，自动化截图生成于 2026-07-26。

四、功能域规划

功能域	覆盖内容	关键规则
认证与账号	登录、注册、找回、验证码、JWT、风控	验证码按用途隔离，失败锁定和权限边界后端复核
读者流通	借阅、归还、续借、预约、罚款、收藏	库存为 1 时并发借阅只允许一个请求成功
内容互动	书评、点赞、回复、举报、留言	审核处置和关键操作写入审计日志
采购物流	需求、认领、运输、到馆、入库	入库状态幂等，库存只补充一次
文件治理	头像、封面、附件、引用、清理	校验扩展名、MIME、大小、文件头和数据权限
通知与中间件	邮件提醒、补偿任务、缓存、事件	Redis/RabbitMQ 关闭时核心业务继续运行

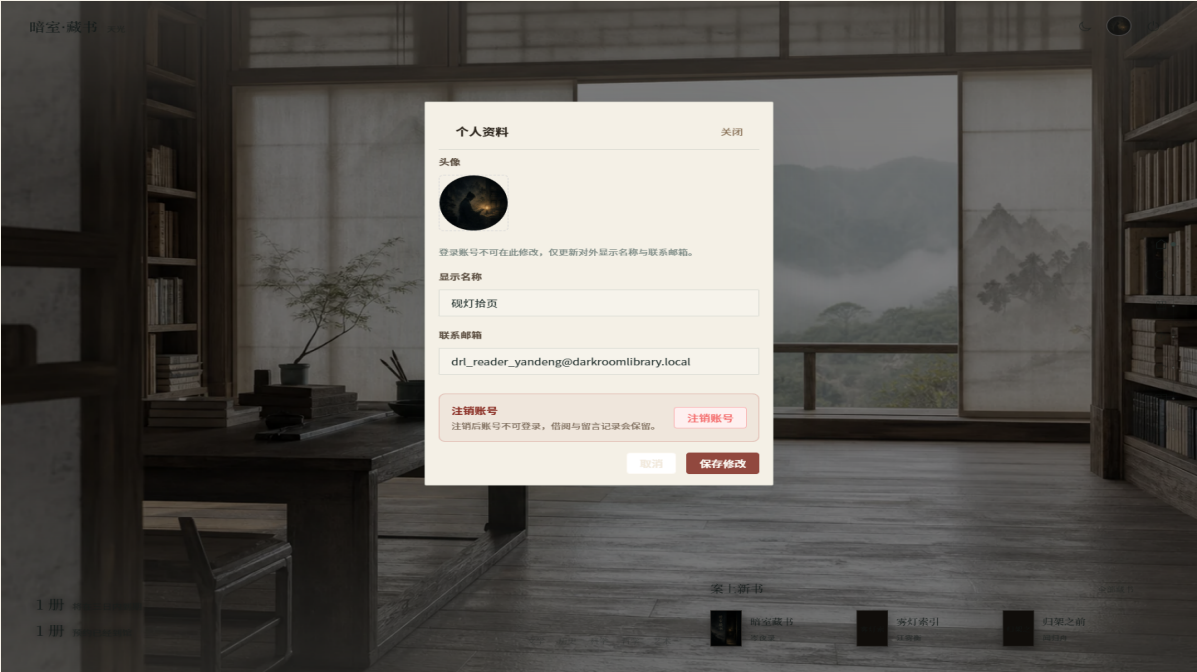


图 2 个人资料弹窗沿用登录页纸面输入逻辑，保存按钮使用封蜡红。

五、技术与数据设计

前端统一采用 Vue 3、Vue Router 4、Element Plus、ECharts、Vite 和 ESLint；不再保留 Vue 2 双轨。后端采用 Spring Boot 3、Spring MVC、MyBatis-Plus 与 Mapper XML 的清晰分层。Controller 负责入口，Service 负责事务和业务规则，Mapper 负责数据访问，适配层负责 Redis、RabbitMQ、邮件和文件系统。

数据库由 19 张业务表和 27 条外键关系构成。库存、借阅、预约、采购入库和文件引用等不变量由数据库约束、条件更新、事务和服务层规则共同守护。Redis 不保存最终库存，RabbitMQ 不决定借阅是否成功。

层次	当前实现	责任
表现层	Vue 3 + Vite + Element Plus	读者端体验和管理端操作界面
接入层	Spring MVC Controller + JWT + 限流	参数校验、身份识别和接口边界
应用层	Service + 事务 + 权限 AOP	状态机、角色边界和业务编排
持久层	MyBatis-Plus + Mapper XML	条件更新、复杂查询和数据访问
事实来源	MySQL 8	业务状态、约束和审计数据
增强设施	Redis / RabbitMQ / 邮件 / 文件存储	加速、异步和补偿，均可降级

六、视觉与交互方向

读者端强调“叩门、入室、接灯、回望”的连续感，使用暗室、宣纸、书卷和灯火意象；管理端强调长期操作的可读性，使用纸面底色、清晰表格、稳定密度和克制强调。隐喻只负责建立气质，不替代按钮的真实功能。

登录页、注册页、个人资料弹窗共享纸面表单和清楚提示；保存、确认等关键操作保持功能文字，个人资料保存按钮使用封蜡红而不是突兀的蓝色。书封面、头像、评论、公告和截图使用项目专用的虚构演示数据。



图 3 读者端暗室首页夜间主题。

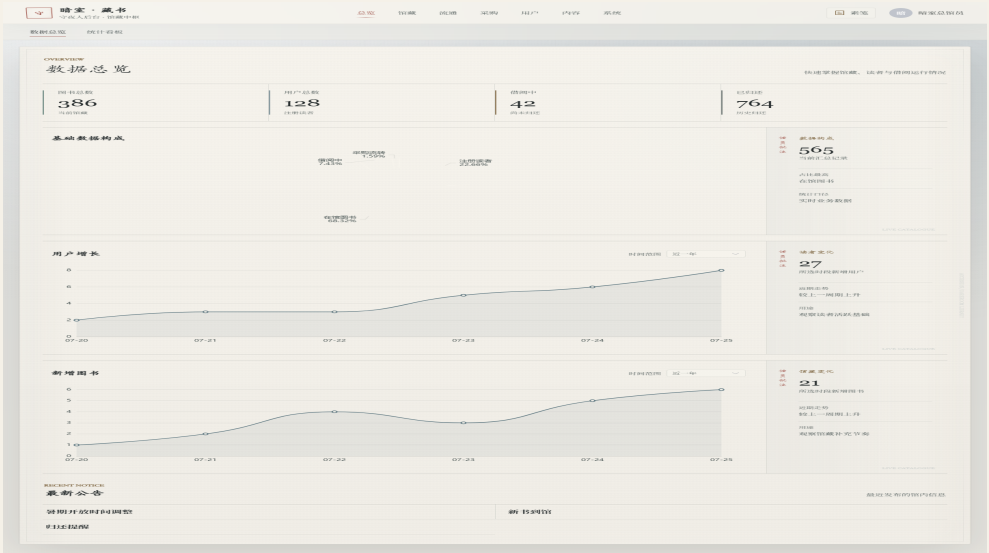


图 4 管理端数据总览，展示信息密度与读者端的差异。

七、实施计划与交付方式

阶段	交付目标	当前状态
业务闭环	借还、预约、互动、采购物流和文件治理	已完成并通过真实流程
数据一致性	事务、条件更新、外键和幂等规则	已完成并通过并发与数据库检查
前端重构	Vue 2 遗留结构迁移到 Vue 3/Vite	已完成，删除旧双轨
验证体系	自动测试、真实 API、F12、数据库和中间件	已完成，证据归档
公开交付	文档、PPT、PDF、Git 基线和 GitHub	已完成审查，进入发布归档

本项目的交付判断不以“代码文件已经写完”为准，而以全链路真实运行、可重复测试、浏览器无异常、数据库不变量成立、文档与实际版本一致为准。

八、当前交付基线

指标	结果
后端测试	225/225 通过
前端单元测试	31/31 通过
五角色真实 API	75 次响应
单实例并发	20 场景 / 393 请求，最大 P95 117 ms
双实例并发	6 场景 / 125 请求，最大 P95 119 ms
浏览器与 F12 等价诊断	86 路由 / 248 次 API，Console/Network/页面异常为 0
数据库完整性	27 条外键关系，孤儿记录和领域违规为 0
工程检查	ESLint、Vite build、npm audit 通过，0 vulnerabilities

完整命令、环境说明和证据路径见 docs/verification-report.md；项目沿革与开源边界见 docs/project-history.md、LICENSE、NOTICE.md。